Instructions:
1. Run task_2.py to generate the tensor database if not already generated.
2. Press Run All (or restart kernel and run all cells).
3. You will be prompted to provide an image ID and a k value.

Task 3: Implement a program which, given an image ID and a value “k”, returns and visualizes the most similar k images based on each of the visual model –you will select the appropriate distance/similarity measure for each feature model.
For each match, also list the corresponding distance/similarity score.

In [ ]:
```
IMG_ID = int(input("Provide an Image ID in the Caltech101 dataset in range [0, 8676]."))

K = int(input("Enter K, the number of similar images K to find for each feature model."))
```

In [ ]:
```
# Initialize dataset, downloads into <git_root>/Code/caltech101 if not present. Untracked by git.

import os
from torchvision.datasets import Caltech101

dataset = Caltech101(root=os.path.abspath(""), download=True)
```

```
Files already downloaded and verified
```

In [ ]:
```
# Find the k most similar images for a given Image ID for each feature model.

image, category = dataset[IMG_ID][0], dataset.categories[dataset[IMG_ID][1]]

if image.mode != 'RGB':
    print("WARNING: The provided image does not have RGB channels, the image will be converted to RGB.")
    image = image.convert('RGB')

print("Input image:\nImage ID: ", IMG_ID)
display(image)
print("Category:", category)
```

```
Input image:
Image ID:  880
```

Category: Leopards

In [ ]:
```
# Resize to 300x100 and partition to grid.

image300x100 = image.resize((300, 100))

from utils.utilities import partition_to_grid

grid = partition_to_grid(image300x100, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

In [ ]:
```
# Color moments (CM10x10).

# Find K most similar images by color moments.

from feature_extractors.color_moments import get_color_vector

color_vector = get_color_vector(grid)

# Load color tensor database into memory.

from utils.utilities import tensor_loader

color_tensor_dict = tensor_loader('color_tensors.pt', os.path.abspath(""))

from utils.utilities import top_k_ranker

# Manhattan distance (City-block distance): This distance can be imagined as the length needed
# to move between two points in a grid where you can only move up, down, left or right. This
# measure is sensitive to differences along each dimension of the feature vectors, capturing
# the total absolute difference between corresponding moments, which makes it great for
# assessing differences in color distribution in terms of absolute deviations. Since it
# does not square the differences like Euclidean distance, it is less susceptible outliers
# when many dimensions differ.
```

```
# We use Earth Mover's Distance (Wasserstein distance) as color moments can be thought
# of as probability distributions over color space. EMD is capable of comparing distributions
# with different shapes and scales, color moments consider the spatial arrangement of colors,
# not just the frequency of colors. EMD takes this spatial information into account by
# considering how color probability mass is transported from one distribution to another.

from scipy.spatial.distance import cityblock

print("Similar images by Color Moments using distance: Manhattan (cityblock)")

top_k_by_color = top_k_ranker(K, color_vector, color_tensor_dict, cityblock)


for i in range(len(top_k_by_color)):

    id = top_k_by_color[i][1]
    distance = top_k_by_color[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Color Moments using distance: Manhattan (cityblock)
1.      Image ID:  880          Distance:  0.0
```

```
2.      Image ID:  980          Distance:  12883.973
```

```
3.      Image ID:  970          Distance:  12894.973
```

4. Image ID: 974 Distance: 12919.006

5. Image ID: 987 Distance: 13368.09

6. Image ID: 925 Distance: 13505.451

7. Image ID: 887 Distance: 13672.375

8. Image ID: 874 Distance: 13687.33

9. Image ID: 876 Distance: 13732.93

10. Image ID: 976 Distance: 13764.921

In [ ]:
```
# Convert to grayscale and partition to grid.

from torchvision.transforms import Grayscale

gs_image = Grayscale()(image300x100)

gs_grid = partition_to_grid(gs_image, num_rows=10, num_cols=10, hor_pixels=30, ver_pixels=10)
```

In [ ]:
```
# Histograms of oriented gradients (HOG).

# Find K most similar images by HOG.

from feature_extractors.HOG import get_hog_vector

hog_vector = get_hog_vector(gs_grid)

# Load hog tensor database into memory.

hog_tensor_dict = tensor_loader('hog_tensors.pt', os.path.abspath(""))

# Correlation similarity: This measures the linear relationship between HOG feature vectors,
# and performs well with features that have a strong linear correlation or dependency in
# their gradient orientation distributions, taking into account directionality.

from scipy.spatial.distance import correlation

print("Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation")

top_k_by_hog = top_k_ranker(K, hog_vector, hog_tensor_dict, correlation)


for i in range(len(top_k_by_hog)):

    id = top_k_by_hog[i][1]
    distance = top_k_by_hog[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by Histogram of Oriented Gradients (HOG) using distance: Correlation
1.      Image ID:  880          Distance:  0.0
```

2. Image ID: 1044 Distance: 0.1108569742828649

3. Image ID: 980 Distance: 0.11876509688290737

4. Image ID: 7406 Distance: 0.12034339240493097

5. Image ID: 944 Distance: 0.12036418127448689

6. Image ID: 1000 Distance: 0.12048191374948791

7. Image ID: 987 Distance: 0.12250058086032545

8. Image ID: 974 Distance: 0.12269712751484674

```
9.      Image ID:  887          Distance:  0.12503799602553078
```

```
10.     Image ID:  911          Distance:  0.12519870864032556
```

In [ ]:
```
# Resize original image to 224x224 and convert to float tensor.

from torchvision.transforms import transforms

image224x224 = image.resize((224, 224))

img_tensor = transforms.Compose([transforms.ToTensor()])(image224x224)
```

In [ ]:
```
# ResNet pre-processing.

# Find K most similar images by each of the 3 layers chosen from ResNet-50.

from feature_extractors.resnet_50 import get_resnet50_feature_vectors

avgpool_vector, layer3_vector, fc1000_vector = get_resnet50_feature_vectors(img_tensor)

# Load each tensor database into memory

avgpool_tensor_dict = tensor_loader('avgpool_tensors.pt', os.path.abspath(""))
```

```
layer3_tensor_dict = tensor_loader('layer3_tensors.pt', os.path.abspath(""))
fc1000_tensor_dict = tensor_loader('fc1000_tensors.pt', os.path.abspath(""))
```

In [ ]:
```
# ResNet-AvgPool-1024.

# Find K most similar images by ResNet-AvgPool-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

from scipy.spatial.distance import cosine

print("Similar images by ResNet-AvgPool-1024 using distance: Cosine")

top_k_by_avgpool = top_k_ranker(K, avgpool_vector, avgpool_tensor_dict, cosine)


for i in range(len(top_k_by_avgpool)):

    id = top_k_by_avgpool[i][1]
    distance = top_k_by_avgpool[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-AvgPool-1024 using distance: Cosine
1.      Image ID:  880          Distance:  0
```

```
2.      Image ID:  980          Distance:  0.02858734130859375
```

3. Image ID: 976 Distance: 0.0936480164527893

4. Image ID: 1022 Distance: 0.10950839519500732

5. Image ID: 898 Distance: 0.11175507307052612

6. Image ID: 996 Distance: 0.1154336929321289

7. Image ID: 922 Distance: 0.11590331792831421

8. Image ID: 998 Distance: 0.11629730463027954

9. Image ID: 896 Distance: 0.11795306205749512

10. Image ID: 886 Distance: 0.11834180355072021

In [ ]:
```
# ResNet-Layer3-1024.

# Find K most similar images by ResNet-Layer3-1024.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-Layer3-1024 using distance: Cosine")

top_k_by_layer3 = top_k_ranker(K, layer3_vector, layer3_tensor_dict, cosine)


for i in range(len(top_k_by_layer3)):

    id = top_k_by_layer3[i][1]
    distance = top_k_by_layer3[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-Layer3-1024 using distance: Cosine
1.      Image ID:  880          Distance:  0
```

```
2.      Image ID:  980          Distance:  0.002772510051727295
```

3. Image ID: 876 Distance: 0.01864105463027954

4. Image ID: 976 Distance: 0.021041393280029297

5. Image ID: 1010 Distance: 0.02151542901992798

6. Image ID: 910 Distance: 0.022718727588653564

7. Image ID: 875 Distance: 0.023593544960021973

8. Image ID: 993 Distance: 0.023832857608795166

9. Image ID: 893 Distance: 0.024298250675201416

10. Image ID: 1022 Distance: 0.025303542613983154

In [ ]:
```
# ResNet-FC-1000.

# Find K most similar images by ResNet-FC-1000.

# Cosine similarity: We use Cosine since the output of the ResNet layer is
# normalized, reducing the importance of magnitude. It is good at discerning
# semantic or structural similarity rather than their absolute feature values.
# It also works well with sparse feature spaces such as those found in CNNs.

print("Similar images by ResNet-FC-1000 using distance: Cosine")

top_k_by_fc1000 = top_k_ranker(K, fc1000_vector, fc1000_tensor_dict, cosine)


for i in range(len(top_k_by_fc1000)):

    id = top_k_by_fc1000[i][1]
    distance = top_k_by_fc1000[i][0]

    print(str(i + 1) + ".", "\tImage ID: ", id, "\t\tDistance: ", distance)
    display(dataset[id][0])
```

```
Similar images by ResNet-FC-1000 using distance: Cosine
1.      Image ID:  880          Distance:  0
```

```
2.      Image ID:  980          Distance:  0.02508002519607544
```

3. Image ID: 976 Distance: 0.1025659441947937

4. Image ID: 999 Distance: 0.10723358392715454

5. Image ID: 922 Distance: 0.10978174209594727

6. Image ID: 1022 Distance: 0.11090725660324097

7. Image ID: 995 Distance: 0.11403411626815796

8. Image ID: 898 Distance: 0.11512356996536255

9. Image ID: 876 Distance: 0.11669296026229858

10. Image ID: 996 Distance: 0.11676293611526489